Autoencoders and Transformer-Based Autoencoders

Instructor: Prof. Olushina Olawale Awe
AIMS Cameroon Deep Learning Course

Why Autoencoders?

Real-world data is:

- High-dimensional
- Noisy
- Redundant

Key Question:

How can we learn compact, meaningful representations of data *without labels*?

Definition of an Autoencoder

Autoencoder

An autoencoder is a neural network trained to reconstruct its input by passing it through a constrained latent representation.

Mapping:

$$x \longrightarrow z \longrightarrow \hat{x}$$

Goal:

$$\hat{x} \approx x$$

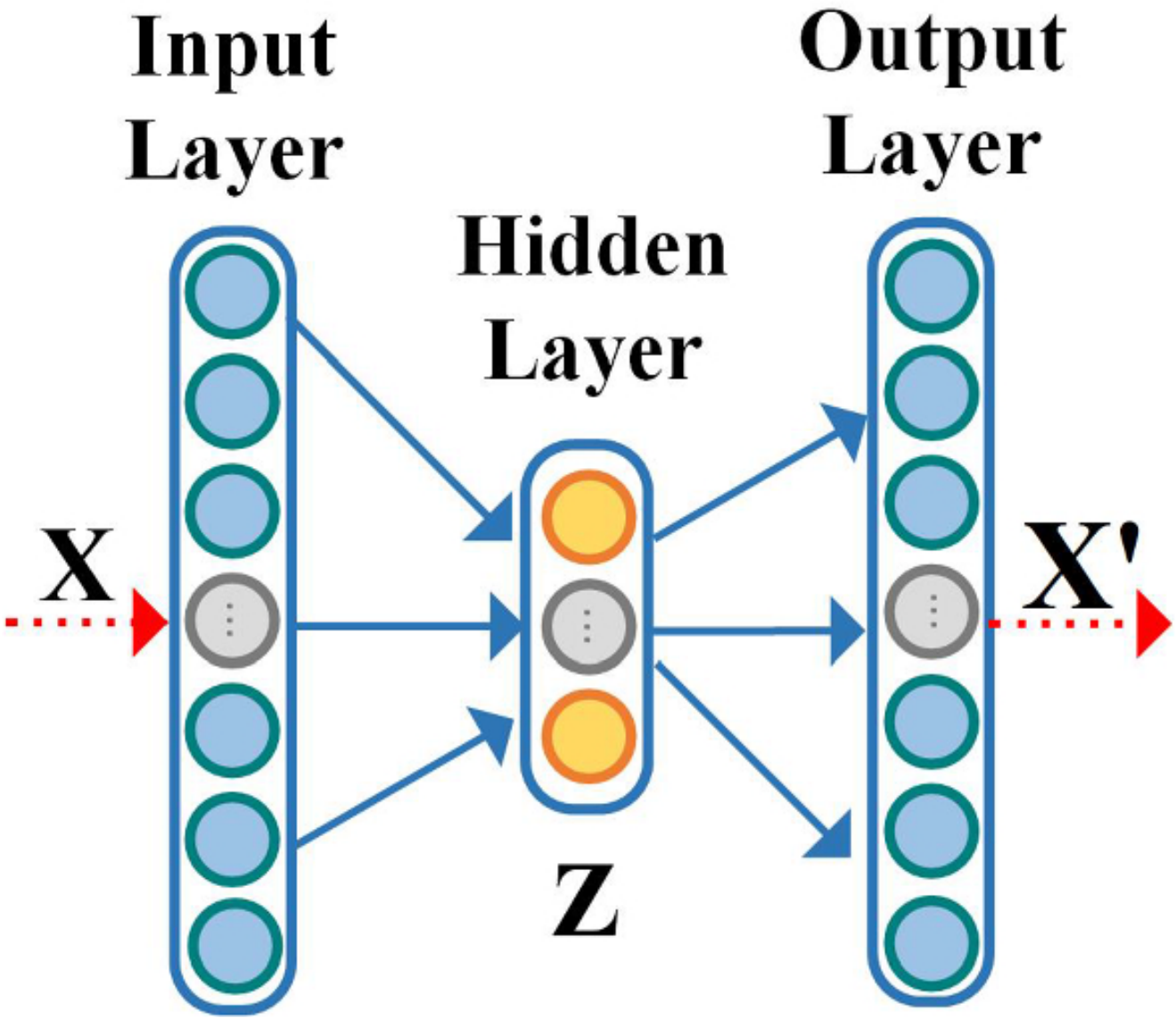


Figure: AE Architecture

What is the Goal of an Autoencoder?

An autoencoder learns a function:

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x}$$

- $x \in \mathbb{R}^d$: input
- $z \in \mathbb{R}^k$: latent representation
- $\hat{x}$: reconstruction of x

Goal: Learn useful representations, not just reconstruction.

Core Components

An autoencoder consists of three parts:

1. **Encoder**: compresses input
2. **Latent space (bottleneck)**: compact representation
3. **Decoder**: reconstructs input

Encoder and Decoder Equations

Encoder:

$$z = f_{\text{enc}}(x) = \sigma(W_e x + b_e)$$

Decoder:

$$\hat{x} = f_{\text{dec}}(z) = \phi(W_d z + b_d)$$

Where:

- σ, ϕ are activation functions
- z is the latent code

Learning Objective

Reconstruction loss (typical):

$$\mathcal{L}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2$$

Training:

- Forward pass: $x \to z \to \hat{x}$
- Backpropagation
- Gradient descent

Avoiding the Identity Function

Without constraints:

$$\hat{x} = x$$

The model learns nothing useful.

Common constraints:

- Bottleneck (low-dimensional z)
- Sparsity
- Noise (denoising autoencoder)
- Regularization

Important Autoencoder Variants

- Denoising Autoencoder
- Sparse Autoencoder
- Variational Autoencoder (VAE)

Each variant enforces structure in the latent space.

Why Classical Autoencoders Struggle

Classical autoencoders:

- Assume fixed-size vectors
- Struggle with sequences
- Capture mostly local structure

Question: How do we autoencode *text*, *time series*, or *long sequences*?

Why Transformers?

Transformers provide:

- Self-attention (global dependency modeling)
- Parallel computation
- Strong sequence representations

This makes them ideal for **sequence autoencoding**.

Encoder–Decoder Transformer Autoencoder

Encoder:

- Self-attention
- Position-wise FFNN
- Produces contextual latent vectors

Decoder:

- Masked self-attention
- Cross-attention to encoder outputs
- Reconstructs input sequence

Transformer-Based Autoencoder

A transformer-based autoencoder uses self-attention to encode a sequence into contextual latent representations and reconstructs the original sequence often via denoising by minimizing reconstruction loss.

What Is the Latent Space Now?

Unlike classical autoencoders:

$$z \neq \text{single vector}$$

Instead:

$$Z = [z_1, z_2, \ldots, z_T]$$

Each z_t:

- Represents token t
- Encodes global context via attention

Denoising with Transformers

Training procedure:

1. Corrupt input (mask or delete tokens)
2. Encode corrupted sequence
3. Reconstruct original tokens

Loss:

$$\mathcal{L} = -\sum_{t \in \mathcal{M}} \log p(x_t \mid x_{\text{visible}})$$

Key Takeaways on TAE

- Autoencoders learn representations via reconstruction
- Transformers enable autoencoding of sequences
- Attention replaces local compression with global context
- BERT is a denoising Transformer autoencoder

Applications of Transformer Autoencoders

- Representation learning
- Text denoising
- Sentence embeddings
- Pretraining for NLP tasks
- Anomaly detection in sequences

The Identity Mapping Problem

If the autoencoder is powerful enough:

$$\hat{x} \approx x$$

- Perfect reconstruction
- No abstraction or structure learned
- Poor generalization

Regularization prevents trivial memorization.

Regularized Autoencoder Objective

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{recon}}(x, \hat{x})}_{\text{Reconstruction}} + \lambda \underbrace{\Omega(\cdot)}_{\text{Regularization}}$$

- Reconstruction term ensures fidelity
- Regularization term enforces structure
- λ controls the strength

1. Bottleneck Regularization

Idea: Force compression by limiting latent dimension.

$$\dim(z) \ll \dim(x)$$

- Encoder must extract key features
- Prevents identity mapping

This is the simplest form of regularization.

Limitations of Bottleneck

- Overcomplete autoencoders still overfit
- High-capacity networks bypass bottleneck

Additional regularization is required in practice.

2. Weight Regularization (L1 / L2)

Penalize large weights:

$$\Omega(W) = \begin{cases} \|W\|_1 & \text{(L1)} \\ \|W\|_2^2 & \text{(L2)} \end{cases}$$

- Controls model complexity
- Encourages smooth mappings

3. Sparse Autoencoders: Intuition

- Only a few neurons should be active at once
- Encourages feature selectivity

Inspired by biological neurons.

Sparse Autoencoders

Average activation of neuron j:

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^{m} z_j^{(i)}$$

KL divergence penalty:

$$\Omega = \sum_j \mathrm{KL}(\rho \,\|\, \hat{\rho}_j)$$

- ρ is a small target value (e.g. 0.05)

4. Denoising Autoencoders: Idea

- Corrupt input with noise
- Train model to reconstruct clean input

$$\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

Why Denoising Works

- Forces robustness
- Learns data manifold
- Reduces sensitivity to noise

Small input changes should not change representations.

5. Contractive Autoencoders: Intuition

- Penalize sensitivity of encoder
- Encourage invariance to perturbations

Contractive Autoencoders

Jacobian penalty:

$$\Omega = \left\| \frac{\partial z}{\partial x} \right\|_F^2$$

- $z = f(x)$ is encoder output
- Frobenius norm measures sensitivity

6. Variational Autoencoder Regularization

VAEs regularize the **latent distribution**.

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

- Prior: $p(z) = \mathcal{N}(0, I)$
- Ensures smooth, continuous latent space

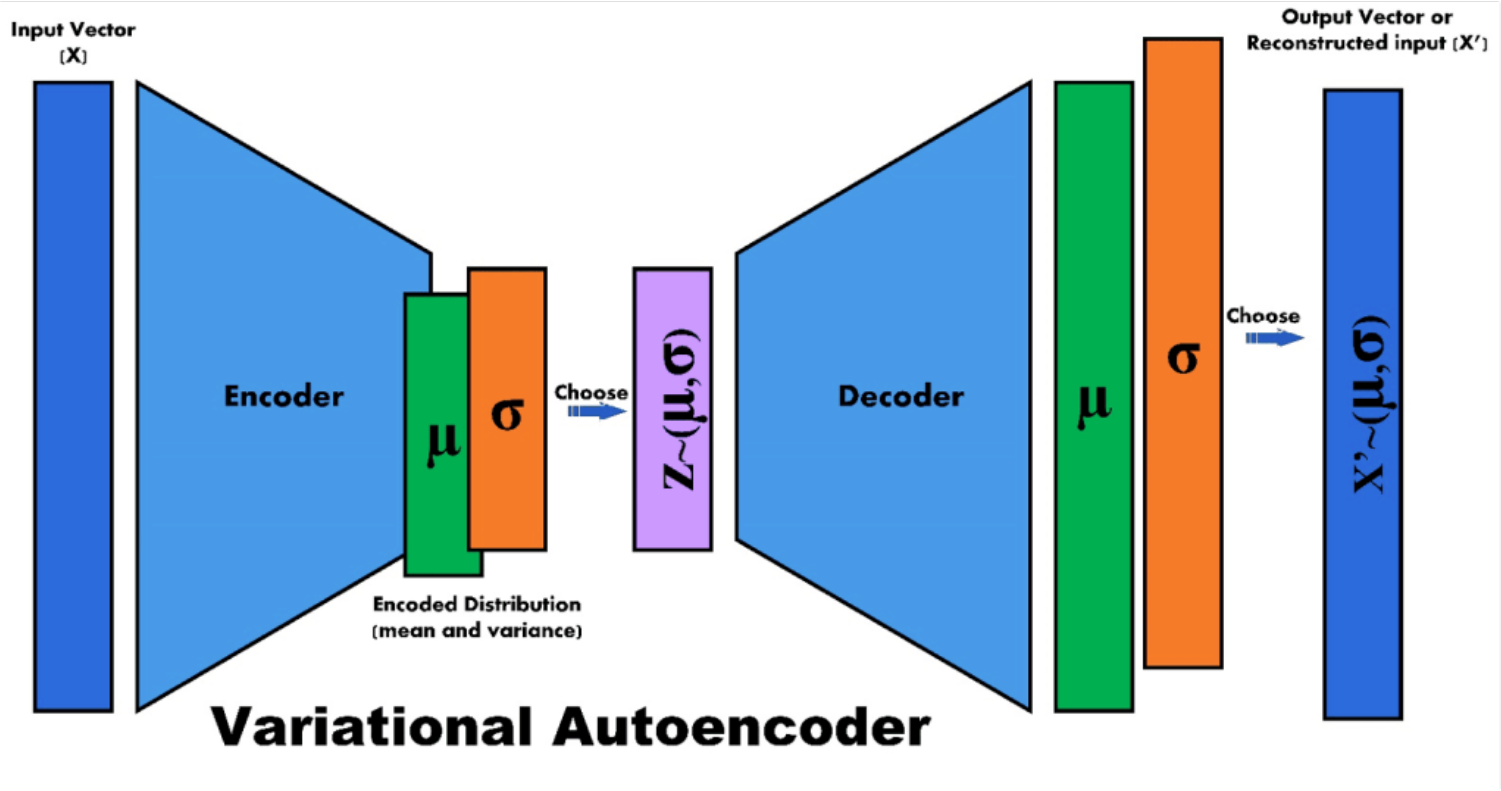

Input Vector
(X)
Encoder
μ
σ
Encoded Distribution
(mean and variance)
Choose
Z~(μ,σ)
Decoder
μ
σ
Choose
X'~(μ,σ)
Output Vector or
Reconstructed input (X')
Variational Autoencoder

Figure: VAE

Comparison of Regularization Methods

Method	Regularizes	Outcome
Bottleneck	Dimension	Compression
L1 / L2	Weights	Smoothness
Sparse AE	Activations	Interpretability
Denoising AE	Input	Robustness
Contractive AE	Jacobian	Invariance
VAE	Distribution	Generativity

Key Message

Regularization in autoencoders defines **what kind of representation** the model is allowed to learn.

- Compression?
- Sparsity?
- Robustness?
- Generative ability?

Applications

- Feature learning
- Anomaly detection
- Image denoising
- Representation learning for downstream tasks

More on Variational Autoencoders

- Standard autoencoders learn **deterministic** latent codes.
- Latent space may be **irregular and disconnected**.
- Cannot reliably **generate new samples**.

VAEs introduce probability to learn a smooth, generative latent space.

Core Idea of a VAE (Intuition)

Key Insight

Instead of mapping each input x to a single latent vector z, VAEs map x to a **distribution over latent variables**.

$$x \rightarrow (\mu(x), \sigma(x)) \rightarrow z \sim \mathcal{N}(\mu, \sigma^2)$$

- Each data point occupies a **region** in latent space.
- Enables smooth interpolation and sampling.

VAE Architecture

- **Encoder**: $q_\phi(z|x)$
- **Decoder**: $p_\theta(x|z)$
- **Prior**: $p(z) = \mathcal{N}(0, I)$

$$x \xrightarrow{\text{Encoder}} (\mu, \log \sigma^2) \;\Rightarrow\; z \xrightarrow{\text{Decoder}} \hat{x}$$

Encoder: Variational Posterior

The encoder approximates the true posterior:

$$q_\phi(z|x) = \mathcal{N}(z; \mu(x), \sigma^2(x))$$

- $\mu(x)$: mean of latent distribution
- $\sigma^2(x)$: variance (uncertainty)
- Learned via neural networks

This is where uncertainty enters the model.

Why Reparameterization is Needed

- Direct sampling blocks backpropagation.
- Gradients cannot flow through random nodes.

Solution: Reparameterization Trick

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

- Randomness moved to ϵ
- μ and σ remain differentiable

Decoder: Generative Model

The decoder defines:

$$p_\theta(x|z)$$

- Reconstructs input from sampled z
- Outputs:
  - Bernoulli parameters (binary data)
  - Gaussian parameters (continuous data)

VAE Objective: ELBO

The VAE maximizes the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathrm{KL}(q(z|x) \,\|\, p(z))$$

Reconstruction Term

$$\mathbb{E}_{q(z|x)}[\log p(x|z)]$$

- Measures how well $\hat{x}$ matches x
- Common choices:
  - Mean Squared Error (MSE)
  - Binary Cross-Entropy (BCE)

Encourages accurate reconstruction.

KL Divergence Term

$$\mathrm{KL}(q(z|x) \,\|\, p(z))$$

- Forces latent distributions toward $\mathcal{N}(0, I)$
- Prevents overfitting and memorization
- Enables sampling from prior

This regularizes the latent space.

Autoencoder vs VAE

Property	**AE**	**VAE**
Latent code	Deterministic	Probabilistic
Generative	No	Yes
Latent space	Irregular	Smooth
Sampling	No	Yes
Theory	Weak	Bayesian

Key Intuition

Think of VAE as:

- Learning **where data lives**
- Learning **how uncertain that location is**
- Learning a latent space you can **sample from**

Applications of VAEs

- Image generation
- Anomaly detection
- Representation learning
- Data augmentation
- Medical imaging

Important VAE Variants

- β-VAE (disentanglement)
- Conditional VAE (controlled generation)
- VAE-GAN (sharper images)
- Time-series VAEs

Key Idea

VAEs unify deep learning and probabilistic modeling, enabling principled generative learning.

Why CNN Autoencoders?

- Fully connected autoencoders ignore spatial structure
- Images have strong local correlations
- CNNs exploit locality and weight sharing

CNN autoencoders preserve spatial meaning.

CNN Autoencoder Architecture

$$x \rightarrow \text{Conv} \rightarrow \text{Pooling} \rightarrow z \rightarrow \text{Upsampling} \rightarrow \text{Conv} \rightarrow \hat{x}$$

- Encoder compresses spatial features
- Decoder reconstructs original image

Encoder: Convolutional Feature Extraction

$$Z^{(l)} = \sigma(X * W^{(l)} + b^{(l)})$$

- Filters learn edges, textures, shapes
- Pooling reduces resolution

Decoder: Reconstruction

- Upsampling or transposed convolution
- Gradually restores spatial resolution

$$\hat{X} = Z * W_{\text{dec}} + b_{\text{dec}}$$

Reconstruction Loss in Autoencoders

Purpose: Measure how well the decoder reconstructs the original input.

General form:

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^{n} \ell(x_i, \hat{x}_i)$$

Common choices:

- **Mean Squared Error (MSE)** (continuous data):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2$$

- **Binary Cross-Entropy (BCE)** (binary / normalized images):

$$\mathcal{L}_{\text{BCE}} = -\sum_{i=1}^{n} \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right]$$

Reconstruction loss enforces fidelity, not generalization , hence the need for regularization

Reconstruction Loss: Intuition

What does reconstruction loss do?

- Penalizes differences between x and $\hat{x}$
- Pushes the decoder to faithfully invert the encoder

Key Limitation

Minimizing reconstruction loss alone may lead to:

- Identity mapping
- Memorization
- Poor latent representations

Reconstruction $\neq$ Representation Learning

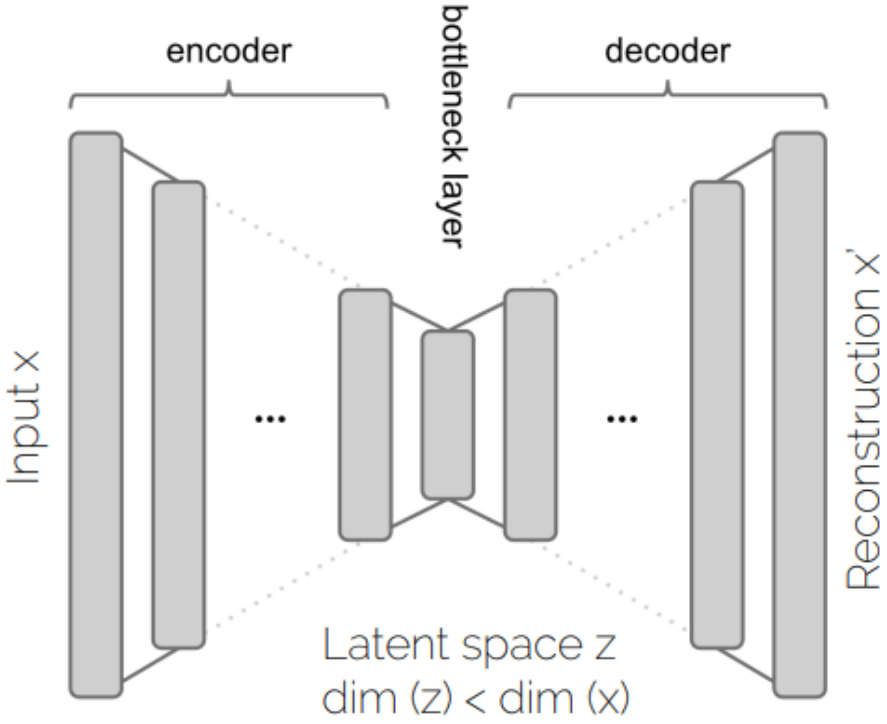
Regularization in CNN Autoencoders

- Bottleneck constraint
- Denoising (input corruption)
- Sparse activations
- Weight decay

Dense AE vs CNN AE

Property	Dense AE	CNN AE
Spatial awareness	No	Yes
Parameter sharing	No	Yes
Image quality	Low	High
Scalability	Limited	Excellent

Autoencoder: training



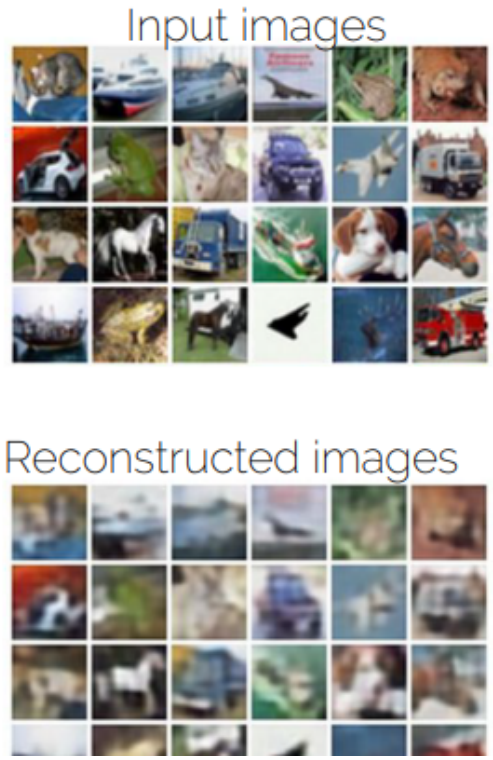


Figure: CNN AE

SegNet

- **Encoder**: normal convolutional filters + pooling
- **Decoder**: Upsampling + convolutional filters
- **Softmax** layer: The output of the soft-max classifier is a K channel image of probabilities where K is the number of classes.

Applications

- Image denoising
- Anomaly detection
- Image compression
- Feature extraction

Take-Home Message

CNN autoencoders are the natural choice for learning representations from images.

References: Autoencoders

- Stanford University. *Deep Learning Tutorial: Autoencoders*. http://ufldl.stanford.edu/tutorial/unsupervised/Autoencoders/
- Chollet, F. *Building Autoencoders in Keras*. Keras Blog. https://blog.keras.io/building-autoencoders-in-keras.html
- TensorFlow Team. *Introduction to Autoencoders*. TensorFlow Tutorials. https://www.tensorflow.org/tutorials/generative/autoencoder
- Bank, D., Koenigstein, N., & Giryes, R. (2020). *Autoencoders*. arXiv preprint arXiv:2003.05991. https://arxiv.org/abs/2003.05991
- Grosse, R. *Lecture on Autoencoders*. University of Toronto. http://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec20.pdf